

tree_example.cpp

```
vector<int> maxdesc, heights;
vector<vector<int>>> adj;

void dist(int node, int h) {
    heights[node] = h;
    maxdesc[node] = h;
    for (auto child : adj[node]) {
        dist(child, h + 1);
        maxdesc[node] = max(maxdesc[node], maxdesc[child]);
    }
}

int pota(int N, vector<int> P) {
    // build tree
    adj.assign(N, vector<int>());
    for (int i = 1; i < N; ++i) {
        adj[P[i]].push_back(i);
    }

    heights.assign(N, 0);
    maxdesc.assign(N, 0);
    dist(0, 0);

    vector<int> source(N + 1, 0), end(N + 1, 0);
    for (int i = 0; i < N; ++i) {
        source[heights[i]]++; // nodi validi fino a una certa altezza h
        end[maxdesc[i] + 1]++; // nodo non piu' valido dalla massima altezza raggiungibile +1
    }

    int curr = 0, keep = 0;
    for (int h = 0; h <= N; ++h) {
        curr += source[h] - end[h];
        keep = max(keep, curr);
    }

    return N - keep;
}

int main() {
    int N;
    cin >> N;
    vector<int> P(N);
    for (int i = 0; i < N; ++i) cin >> P[i];
    cout << pota(N, P) << endl;
}
```

hashing.cpp

```
const ll p = 31;

int main() {
    int N; cin >> N;
    string S; cin >> S;

    // precompute hashes and pows
    vector<ll> hashes(N+1);
    vector<ll> pows(N+1,1);
    for(int i = 0; i < N; i++) {
        hashes[i + 1] = hashes[i] * p + S[i] - 'a'; // partial hash
        pows[i + 1] = p * pows[i]; // increment exp
    }

    vector<vector<int>>> divs(N+1);

    for(int i = 2; i <= N; i++) {
        if(!divs[i].empty()) continue;
        for(int j = i; j <= N; j += i)
            divs[j].push_back(j / i);
    }

    auto range_hash = [&](int l, int r) {
        return hashes[r] - (hashes[l] * pows[r - l]);
    };

    int Q;
    cin >> Q;
    while(Q--) {
        int l, r;
        cin >> l >> r;
        r++;

        bool pal = false;
```

```

        for(auto div: divs[r-l]) {
            if(div == r - l) continue;
            ll h1 = range_hash(l, r - div);
            ll h2 = range_hash(l + div, r);

            if(h1 == h2) { pal = true; break; }
        }
        if(pal) cout << "YES" << endl;
        else cout << "NO" << endl;
    }
}

```

fast_exp.cpp

```

// fast modular pow
ll fast_exp(ll base, ll exp) {
    ll res = 1;
    for(; exp; exp >>= 1) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
    }
    return res;
}

```

fenwick.py

```

def fenwick_sum(tree, i): # return sum(t[:i])
    if i == len(tree)-1: return tree[0]
    res = 0
    while i > 0:
        res += tree[i]
        i -= i & -i
    return res
def fenwick_add(tree, i, v=1): # t[i] += v
    tree[0] += v
    i += 1
    while i < len(tree):
        tree[i] += v
        i += i & -i

```

```

if __name__ == "__main__":
    for _ in range(int(input())):
        n = int(input())
        t = list(map(int, input().split()))
        p = [None] * n
        for i in range(n):
            p[t[i]-1] = i
        result = 0
        tree = [0] * (n+1)
        for i in reversed(range(n)):
            if i+1 < n:
                if p[i] < p[i+1]:
                    v = fenwick_sum(tree, p[i+1]) \
                        - fenwick_sum(tree, p[i])
                else:
                    v = fenwick_sum(tree, n) \
                        - fenwick_sum(tree, p[i]) \
                        + fenwick_sum(tree, p[i+1])
                result += (i+1) * v
            fenwick_add(tree, p[i])
        print(result)

```

sparse.cpp

```

// Precomputed minimum values for each range [i, j]
vector<vector<int>>> minInRange;

// Recursive function to compute dp[i][k]
void computeDP(int i, int k, int l, int r, int optl, int optr, vector<vector<long long>>& dp) {
    if (l > r) return;

    int mid = (l + r) / 2;
    pair<long long, int> best = {LLONG_MIN, -1};

    for (int j = optl; j <= min(mid, optr); j++) {
        long long val = dp[j][k-1] + minInRange[j][mid];
        if (val > best.first) {
            best = {val, j};
        }
    }

    dp[mid][k] = best.first;
    computeDP(i, k, l, mid-1, optl, best.second, dp);
    computeDP(i, k, mid+1, r, best.second, optr, dp);
}

int main() {
    int N, K;
    ifstream cin("input0.txt");

```

```

cin >> N >> K;

vector<int> T(N);
for (int i = 0; i < N; ++i)
    cin >> T[i];

// Compute the minimum values for each range [i, j]
vector<vector<long long>> dp(N+1, vector<long long>(K+1, LLONG_MIN));
dp[0][0] = 0;

for (int k = 1; k <= K; ++k) computeDP(N, k, 1, N, 0, N-1, dp);

cout << dp[N][K] << endl;
}

```

priority_queue.cpp

https://cp-algorithms.com/graph/dijkstra_sparse.html#priority_queue
 // find min/max element/path using priority queue, can be also used to find shortest path with best conditions (dijkstra) on a sparse graph

```

int N, M, C_0, P[MAXN], i;
vector<pair<int, int>> adj[MAXN];

int main()
{
    cin >> N >> M >> C_0;
    for (i = 0; i < N; i++)
        cin >> P[i];

    for (i = 0; i < M; i++)
    { // build adj
        int a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }

    adj[N - 1].push_back({N, 0});

    vector<vector<int>> dst(N + 1, vector<int>(MAXC, 2e9));
    vector<vector<bool>> visited(N + 1, vector<bool>(MAXC, false));
    priority_queue<tuple<int, int, int>> q;

    q.push({0, 0, C_0});
    dst[0][C_0] = 0;

    while (!q.empty())
    {
        auto [cost, node, coins] = q.top();
        q.pop();

        if (visited[node][coins])
            continue;

        visited[node][coins] = true;
        cost *= 1; // -1 if min queue

        for (auto [curr, coins] : adj[node])
        {
            // implement logic
            for (int hints = 0; hints <= coins; hints++)
            {
                int new_dst = max(cost, P[node] >> hints); // max between taking it or spending a coin (/2
difficulty)

                int new_coins = coins - hints + coins;
                cout << new_coins << endl;
                if (new_dst < dst[curr][new_coins])
                {
                    dst[curr][new_coins] = new_dst;
                    q.push({-new_dst, curr, new_coins});
                }
            }
        }

        cout << *min_element(dst[N].begin(), dst[N].end()) << endl;
        return 0;
    }
}

```

weighted_graph.cpp

```

int N, M, C_0, a, b, c, i;
int P[MAXN];
vector<pair<int, int>> adj[MAXN];
int main()
{
    assert(3 == scanf("%d %d %d", &N, &M, &C_0));
    for (i = 0; i < N; i++)
    {
        assert(1 == scanf("%d", &P[i]));
    }
}

```

```

}
for (i = 0; i < M; i++)
{
    assert(3 == scanf("%d %d %d", &a, &b, &c));
    adj[a].push_back({b, c});
}

adj[N - 1].push_back({N, 0});

vector<vector<int>> dist(N + 1, vector<int>(MAXC, 2e9));
priority_queue<tuple<int, int, int>> q;
vector<vector<bool>> visited(N + 1, vector<bool>(MAXC, false));

q.push({0, 0, C_0});
dist[0][C_0] = 0;

while (!q.empty())
{
    auto [cost, source, curr_coins] = q.top();
    q.pop();

    if (visited[source][curr_coins])
        continue;
    visited[source][curr_coins] = true;
    cost *= -1;

    for (auto [dest, coins] : adj[source])
    {
        for (int hint = 0; hint <= curr_coins; hint++)
        {
            int new_dist = max(cost, P[source] >> hint); // take the puzzle or cost / 2 spending a coin
            int new_coins = curr_coins - hint + coins;

            if (new_dist < dist[dest][new_coins])
            {
                dist[dest][new_coins] = new_dist;
                q.push({-new_dist, dest, new_coins});
            }
        }
    }
}
printf("%d\n", *min_element(dist[N].begin(), dist[N].end())); // print the result
return 0;
}

```

dp_iter.cpp

```

int N, K;
vector<int> T;

int main() {
    cin >> N >> K;
    int a;

    for (int i = 0; i < N; ++i) {
        cin >> a;
        T.push_back(a);
    }

    int M;
    cin >> M;
    int b;
    vector<set<int>> known(K+1);
    for (int i = 0; i < M; ++i) {
        cin >> a >> b;
        known[a].insert(b);
    }

    int P = 0;

    vector<int> dp(N + 1, -1e9);
    dp[N] = 0;

    //bottom up iterative approach
    for(int i = N-1; i >= 0; i--) {
        //cout << i << endl;
        if(T[i] == K+1) continue;
        if(i+1 == N or (i+1 < N and known[T[i]].count(T[i+1]))) dp[i] = max(dp[i],1+dp[i+1]);
        if(i+2 == N or (i+2 < N and known[T[i]].count(T[i+2]))) dp[i] = max(dp[i],1+dp[i+2]); // skip trick
    }
    cout << max({dp[0],dp[1],0}) << endl;
}

```

dp_bitmask.cpp

```

int main() {

```

```

int n, m;
cin >> n >> m;
vector<vector<int>> cost_col(n, vector<int>(n, 0));
for (int k = 0; k < m; k++) {
    int i, j, a, b;
    cin >> i >> j >> a >> b;
    i--; j--;
    cost_col[j][i] = a;
    cost_col[i][j] = b;
}

vector<int> dp(1 << n, 2e9);
dp[0] = 0;
for (int s = 1; s < (1 << n); s++) {
    for (int j = 0; j < n; j++) {
        if (s & (1 << j)) {
            int s0 = s ^ (1 << j);
            int total = 0;
            int temp = s0;
            while (temp) {
                int i = __builtin_ctz(temp);
                total += cost_col[j][i];
                temp -= temp & -temp;
            }
            if (dp[s] > dp[s0] + total) {
                dp[s] = dp[s0] + total;
            }
        }
    }
}
cout << dp[(1 << n) - 1] << endl;
}

```

segtree.cpp

// source: <https://www.geeksforgeeks.org/segment-tree-in-cpp/>

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define INF ((long long)1e18)
```

```
#define _ << " " <<
```

```
struct Node {
    long long sum, max_sub, max_pref, max_suff;
    int nb, ne;
```

```
    Node *lc, *rc;
};
```

```
class Tree {
    Node *tree, *root;
    int nodes;

    Node* build(int arr[], int nb, int ne) {
        if (nb + 1 == ne) {
            tree[nodes] = {arr[nb], max(0, arr[nb]), max(0, arr[nb]), max(0, arr[nb]), nb, ne, NULL, NULL};
            return &tree[nodes++];
        }
        Node *lc = build(arr, nb, (nb + ne) / 2);
        Node *rc = build(arr, (nb + ne) / 2, ne);
        tree[nodes] = {
            lc->sum + rc->sum,
            max({lc->max_sub, rc->max_sub, lc->max_suff + rc->max_pref}),
            max(lc->max_pref, lc->sum + rc->max_pref),
            max(lc->max_suff + rc->sum, rc->max_suff),
            nb, ne, lc, rc
        };
        return &tree[nodes++];
    }
}
```

```
void change(int x, int y, Node* node) {
    if (node->nb > x || node->ne <= x) return;
    if (node->nb + 1 == node->ne) {
        node->sum = y;
        node->max_sub = node->max_pref = node->max_suff = y;
        return;
    }
    change(x, y, node->lc);
    change(x, y, node->rc);

    node->sum = node->lc->sum + node->rc->sum;
    node->max_pref = max(node->lc->max_pref, node->lc->sum + node->rc->max_pref);
    node->max_suff = max(node->lc->max_suff + node->rc->sum, node->rc->max_suff);
    node->max_sub = max({node->lc->max_sub, node->rc->max_sub, node->lc->max_suff + node->rc->max_pref});
}
```

```
Node query(int x, int y, Node* node) {
    if (node->nb >= y || node->ne <= x) {
        return {0, -INF, -INF, -INF, node->nb, node->ne, NULL, NULL};
    }
}
```

```

    }
    if (x <= node->nb && node->ne <= y) {
        return *node;
    }

    Node lc = query(x, y, node->lc);
    Node rc = query(x, y, node->rc);

    return {
        lc.sum + rc.sum,
        max({lc.max_sub, rc.max_sub, lc.max_suff + rc.max_pref}),
        max(lc.max_pref, lc.sum + rc.max_pref),
        max(lc.max_suff + rc->sum, rc.max_suff),
        node->nb, node->ne, NULL, NULL
    };
}

public:
    long long query(int x, int y) {
        return query(x, y, root).max_sub;
    }

    void change(int x, int y) {
        change(x, y, root);
    }

    Tree(vector<int> arr) {
        tree = (Node*)malloc(1000000 * sizeof(Node));
        nodes = 0;
        root = build(arr.data(), 0, arr.size());
    }
};

int N, Q;
vector<int> V1, V2;

int main() {
    cin >> N >> Q;
    V1.resize(N);
    V2.resize(N);
    for (int i = 0; i < N; i++) {
        cin >> V1[i];
        if (i & 1) V1[i] *= -1;
        V2[i] = -V1[i];
    }

    Tree seg1(V1), seg2(V2);

    for (int i = 0; i < Q; i++) {
        int t;
        cin >> t;
        if (t == 1) {
            int a, b;
            cin >> a >> b;
            if (a & 1) {
                seg1.change(a - 1, b);
                seg2.change(a - 1, -b);
            } else {
                seg1.change(a - 1, -b);
                seg2.change(a - 1, b);
            }
        } else {
            int l, r;
            cin >> l >> r;
            cout << max(seg1.query(l - 1, r), seg2.query(l - 1, r)) << endl;
        }
    }

    return 0;
}

```

[stack.cpp](#)

```

const int INF = 1e9;

int main() {

    int N; int max_h = -1; int max_idx = -1;

    if (!(cin >> N)) return 0;

    vector<int> H(N);

    for (int i = 0; i < N; ++i) {

        cin >> H[i];
    }
}

```

```

        if (H[i] > max_h) { // Troviamo subito la torre più alta per escluderla dopo

            max_h = H[i];

            max_idx = i;

        }

    }

    vector<int> left_greater_idx(N, -1);

    vector<int> right_greater_idx(N, -1);


    stack<int> s_left; // prev greater element (right)

    for (int i = 0; i < N; ++i) {

        while (!s_left.empty() && H[s_left.top()] < H[i]) s_left.pop();

        if (!s_left.empty()) left_greater_idx[i] = s_left.top();

        s_left.push(i);

    }

    stack<int> s_right; // next greater element (right)

    for (int i = N - 1; i >= 0; --i) {

        while (!s_right.empty() && H[s_right.top()] < H[i]) s_right.pop();

        if (!s_right.empty()) right_greater_idx[i] = s_right.top();

        s_right.push(i);

    }

    long long total_score = 0;

    for (int i = 0; i < N; ++i) {

        if (i == max_idx) continue; // La torre più alta va ignorata

        int dist_left = INF;

        int dist_right = INF;

        if (left_greater_idx[i] != -1) dist_left = i - left_greater_idx[i]; // Se esiste una torre + alta a sinistra,
calcolo la distanza

        if (right_greater_idx[i] != -1) dist_right = right_greater_idx[i] - i; // a destra

        total_score += min(dist_left, dist_right);

    }

    cout << total_score << endl;

}

```